



University of
Nottingham

UK | CHINA | MALAYSIA

A large, high-resolution image of the Earth as seen from space, showing the Western Hemisphere with North and South America. The image is framed by a thin white border.

COMP-2024 / COMP-2039
**Artificial Intelligence
Methods**

Lecture 4b
Parameter Setting Issues

IDENTIFY

1. Move Acceptance
2. Simulated Annealing
3. **Parameter Setting Issues**



ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM



**University of
Nottingham**

UK | CHINA | MALAYSIA

Parameter Setting Issues and Tuning Methods



Parameter Setting Issues and Tuning Methods

- **Parameter setting** in metaheuristics refers to the process of selecting and configuring the values of parameters that control the behavior of a metaheuristic algorithm. These parameters significantly affect the algorithm's performance, efficiency, and ability to find optimal or near-optimal solutions.
- Metaheuristics rely on parameters to manage aspects like *exploration, exploitation, convergence rate, and computational resource usage*.
- Parameters must strike a *balance between exploration* (searching new areas of the solution space) *and exploitation* (refining known good areas). The best parameters are often *problem-dependent*, making tuning a critical step for practical applications.



Metaheuristics

Local Search	•	[Kirkpatrick, 1983]	Simulated Annealing (SA)
	•	[Glover, 1986]	Tabu Search (TS)
	•	[Voudouris, 1997]	Guided Local Search (GLS)
	•	[Stutzle, 1999]	Iterated Local Search (ILS)
	•	[Mladenovic, 1999]	Variable Neighborhood Search (VNS)
Population-based	•	[Holland, 1975]	Genetic Algorithm (GA)
	•	[Smith, 1980]	Genetic Programming (GP)
	•	[Goldberg, 1989]	Genetic and Evolutionary Computation (EC)
	•	[Moscato, 1989]	Memetic Algorithm (MA)
	•	[Kennedy and Eberhart, 1995]	Particle Swarm Optimisation (PSO)
Constructive	•	[Dorigo, 1992]	Ant Colony Optimisation (ACO)
	•	[Resende, 1995]	Greedy Randomized Adaptive Search Procedure (GRASP)

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Paul & Dr Chen ZhiYuan, UNM



Metaheuristics

Examples of Parameters

Local Search	• SA	T_0 (initial temperature), α (cooling rate)
	• TS	Tabu list size (tabu tenure)
	• GLS	λ (intensification control), a (coefficient)
	• ILS	<u>perturbation, perturbation strength</u>
	• VNS	k_{min} , k_{max} (smallest, largest neighbourhood size)
Population-based	• GA	population size, mutation probability, <i>mutation</i>
	• GP	
	• EC	
	• MA	
	• PSO	number of particles, V_{max} (maximum velocity)
Constructive	• ACO	weight of pheromone, evaporation rate
	• GRASP	restricted candidate list parameter



Parameter Types

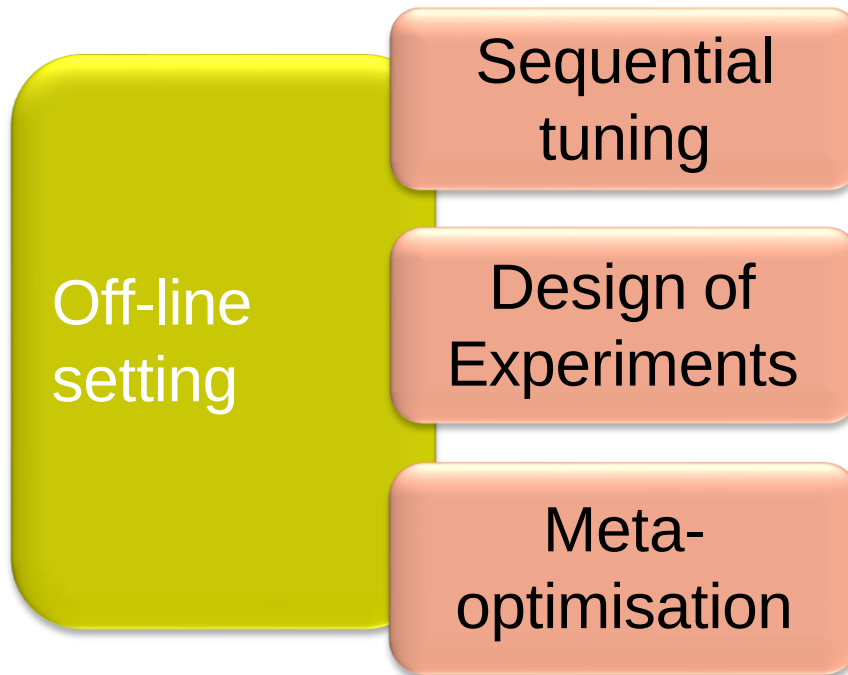
- Categorical/symbolic/structural parameters
 - Choice of initialisation method, choice of mutation,...
- Ordinal parameters
 - Neighbourhoods (e.g., small, medium, large),...
- Numerical/behavioural parameters
 - integer, real-valued, ...
 - population sizes, evaporation rates,...
 - values may depend on the setting of categorical or ordinal parameters.



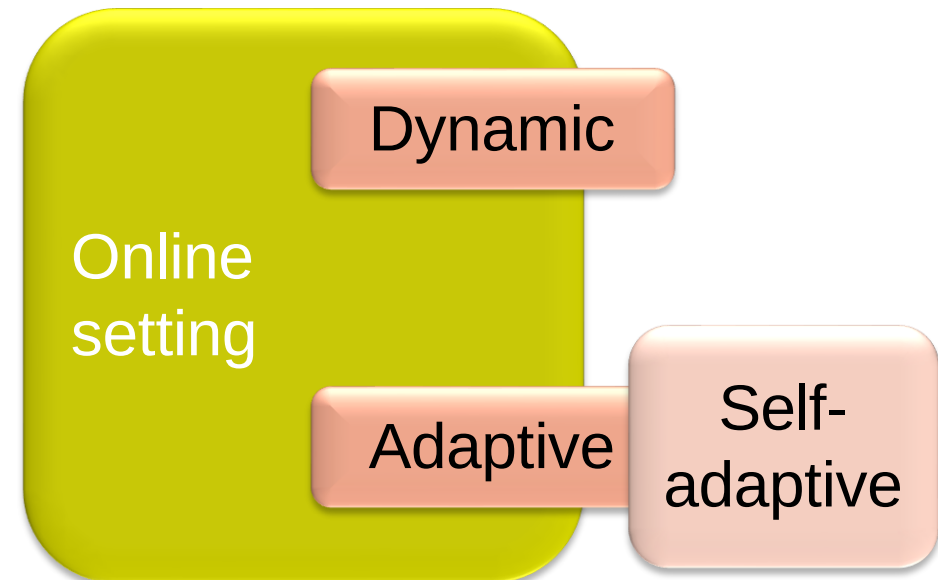
A Classification

Parameter Setting

Parameter Tuning



Parameter Control





Parameter Setting Methods

- **Parameter tuning**: Finding the best *initial settings* for a set of parameters **before** the search process starts (*off-line*). The parameters are fixed before the execution of the algorithm. Commonly used when the problem is well-defined and consistent. E.g., fixing the mutation strength in ILS, mutation probability in genetic algorithms, etc.
 - The *initial parameter setting* influences the performance of a metaheuristic.
- **Parameter control**: Managing the settings of parameters **during** the search process (*online*) (dynamic, adaptive, self-adaptive). E.g., changing the mutation strength in ILS, changing the mutation probability in genetic algorithms during the search process.
 - Controlling parameter setting could yield a system which is not sensitive to its initial setting.



Parameter Tuning

- **Sequential tuning (Trial and Error):** Manually experimenting with different parameter values. Parameters are adjusted iteratively, usually based on trial and error or empirical testing, to find the best settings.
- **Design of Experiments (DOE):** Using systematic statistical techniques to determine the best parameter settings.
- **Meta-optimization:** Using one optimization algorithm to tune the parameters of another algorithm (e.g., tuning a Genetic Algorithm using Particle Swarm Optimization).



Parameter Control

- **Dynamic:** Parameters are changed according to a predefined schedule or function (e.g., decay rates, periodic adjustments).
- **Adaptive:** Parameters are modified based on feedback or performance metrics observed during execution.
- **Self-adaptive:** Parameters are encoded within the algorithm and are adjusted automatically, often using mechanisms like evolutionary strategies or learning systems.



University of
Nottingham

UK | CHINA | MALAYSIA

Parameter Tuning



Parameter Tuning Methods

- **Traditional approaches**

- Use of an arbitrary setting (e.g., $\phi = 0$ and $\delta = 80$)
- Trial& error with settings based on intuition
- Use of theoretical studies
- A mixture of above

- **Sequential tuning:** fix parameter values successively (e.g., fix fixing $\delta = 20$ and tune ϕ that is try $\{0, 0.3, 0.5, 0.8, 1.0\}$, then fixing the best setting for ϕ from the previous trials and tune δ that is try $\{20, 40, 50, 60, 80\}$).

- **Design of experiments**

- **Meta-optimisation:** use a metaheuristic to obtain “optimal” parameter settings.

Example: Assume

$\phi \in \{0, 0.3, 0.5, 0.8, 1.0\}$ and
 $\delta \in \{20, 40, 50, 60, 80\}$

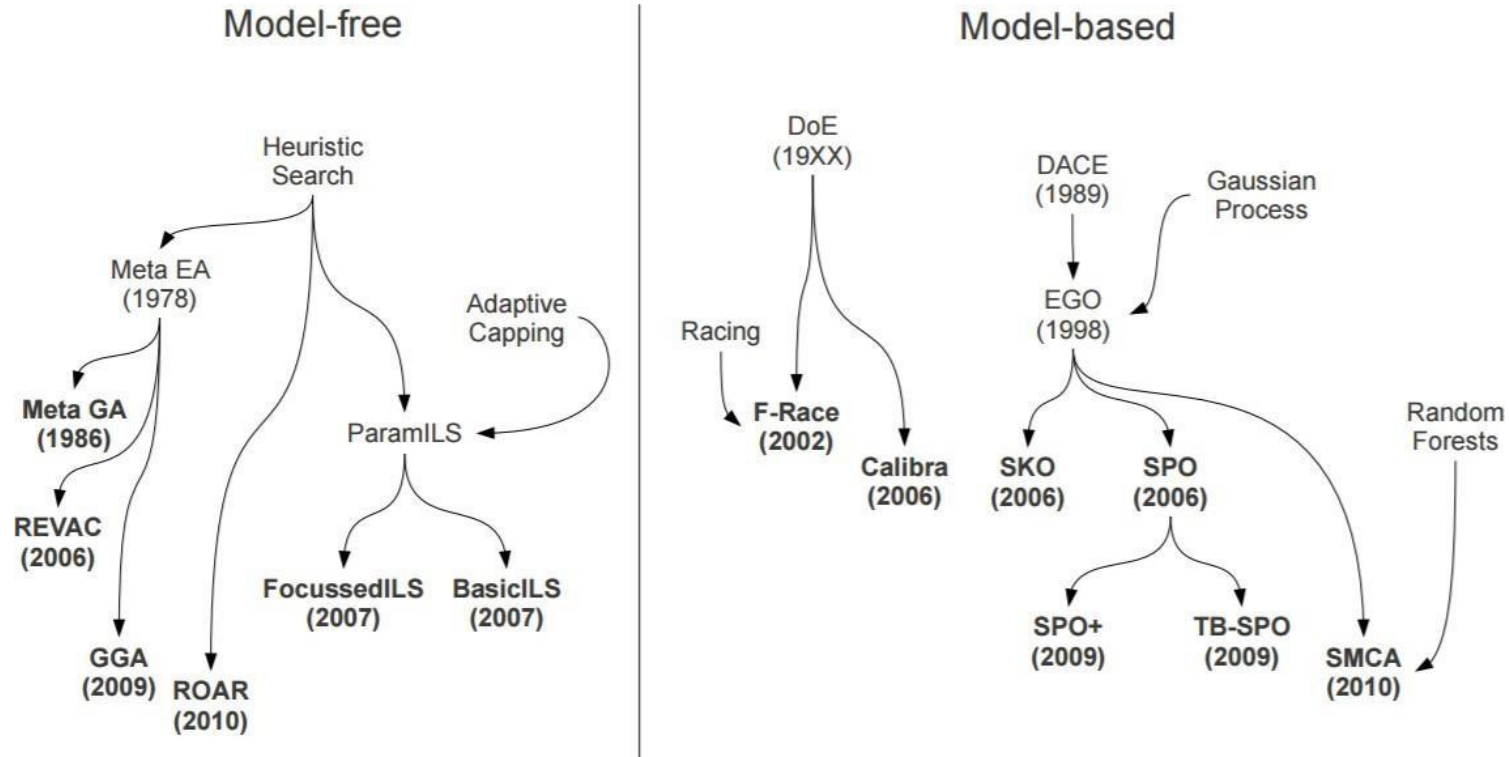


Automated Parameter Tuning

- **Automated parameter tuning** involves systematically finding the best parameter settings without requiring extensive manual effort.
- **Model-free** methods do not rely on an underlying model of the algorithm's performance. Instead, they use direct experimentation to explore the parameter space.
- **Model-based** methods utilize a statistical or mathematical model to predict the performance of different parameter settings, allowing for more efficient exploration.



Automated Parameter Tuning



Source: http://www.mff.cuni.cz/veda/konference/wds/proc/pdf10/WDS10_109_i1_Dobslaw.pdf

I-race (download: <http://iridia.ulb.ac.be/irace/>)

ParamILS (download: <http://www.cs.ubc.ca/labs/beta/Projects/ParamILS/>)

SPOT (download: <https://cran.r-project.org/web/packages/SPOT/index.html>)



Model-free

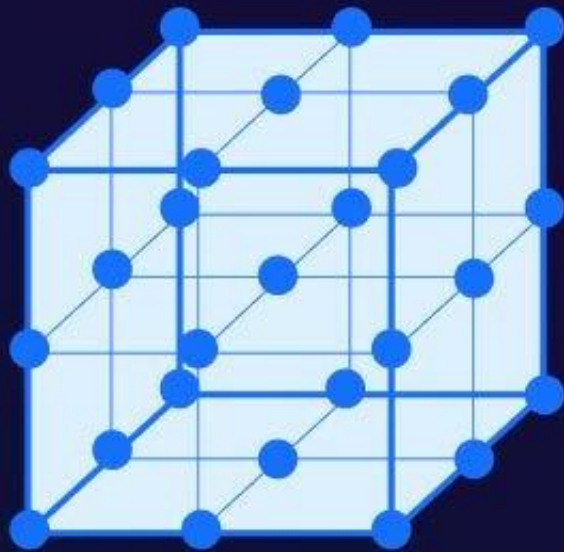
- **Heuristic Search:** This involves using heuristic methods to explore different parameter settings and identify promising regions in the parameter space.
- **Adaptive Capping:** This technique adjusts parameter values dynamically, often based on performance feedback, to ensure that parameter settings remain effective throughout the optimization process.
- **Meta-EA (Evolutionary Algorithm):** This refers to using evolutionary algorithms to optimize the parameters of other algorithms.
- **GGA (Genetic Algorithm):** This method applies genetic algorithms specifically for parameter tuning.
- **ROAR (Robustness Optimization Algorithm for Randomization):** This focuses on improving the robustness of algorithms by tuning their parameters.
- **FocussedLS and BasicLS:** These are specific implementations that utilize local search methods for tuning parameters.



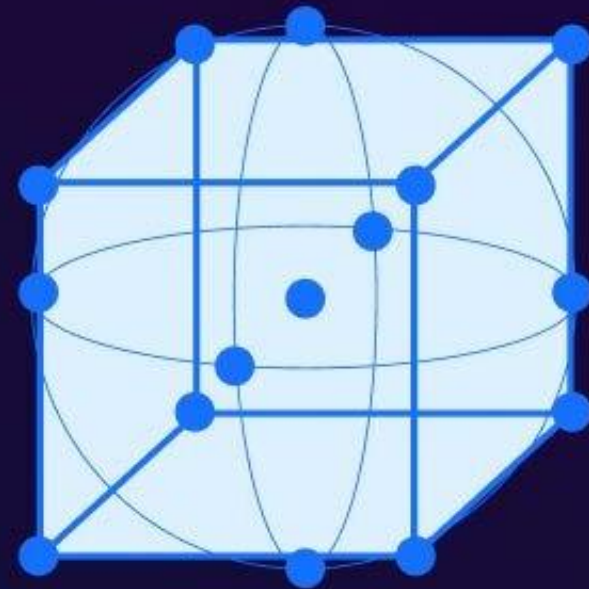
Model-based

- **Design of Experiments (DoE):** This systematic approach helps in understanding how different parameters influence algorithm performance by conducting controlled experiments.
- **DACE (Design and Analysis of Computer Experiments):** This method uses statistical techniques to model and analyze the performance of algorithms based on experimental data.
- **Racing:** A technique that compares different parameter configurations and discards less promising ones early in the tuning process to focus resources on the most promising configurations.
- **F-Race (Focused Race):** A specific racing algorithm designed to efficiently evaluate and select the best parameter settings.
- **SKO (Surrogate-based Knowledge Optimization):** This approach uses surrogate models to approximate the performance of the algorithm based on limited evaluations.
- **SPO (Surrogate-based Parameter Optimization):** Similar to SKO, but focuses specifically on tuning parameters using surrogate models.
- **TB-SPO (Tree-based Surrogate Parameter Optimization):** This method enhances SPO by using tree-based models for parameter tuning.
- **SMCA (Surrogate Model-based Control Algorithm):** This employs surrogate models to guide the tuning process.

What is Design of Experiments (DOE)?



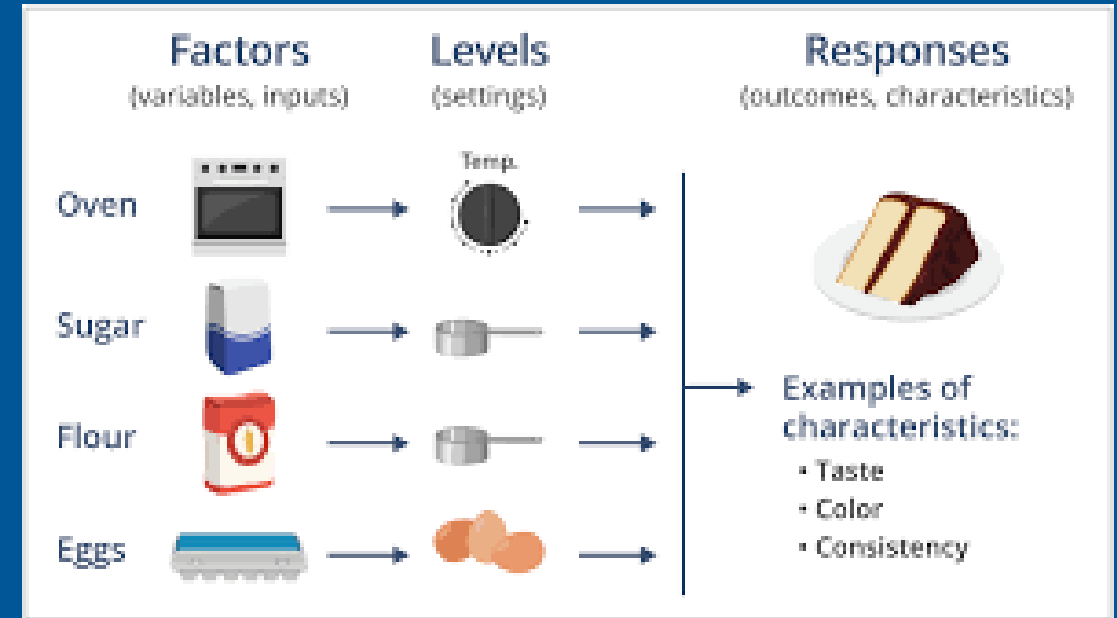
How can we maximize insight and minimize experimentation?





Design of Experiments (DoE)

- Design of Experiments (DOE) is a structured, systematic approach to investigating the relationships between input variables (factors) and their effects on output variables (responses) in a process or system. It helps identify *optimal settings* for factors to achieve desired outcomes, making it highly useful for parameter tuning in optimization and metaheuristic algorithms.





Design of Experiments (DoE)

- “A systematic method (controlled experiments) to determine the relationship between controllable and uncontrollable **factors** (inputs to the process, variables) affecting a process (e.g., running of an algorithm), their **levels** (settings) and the **response** (output) of that process (e.g., quality of solutions obtained – performance of an algorithm).” (Fisher 1926, 1935).
- Important *outcomes* are measured and analysed to determine the *factors* and their *settings* that will provide the best overall outcome.
 - E.g., Two factors; $\phi \in \{0, 0.3, 0.5, 0.8, 1.0\}$ and $\delta \in \{20, 40, 50, 60, 80\}$, each with 5 levels – at least 5^2 runs required.

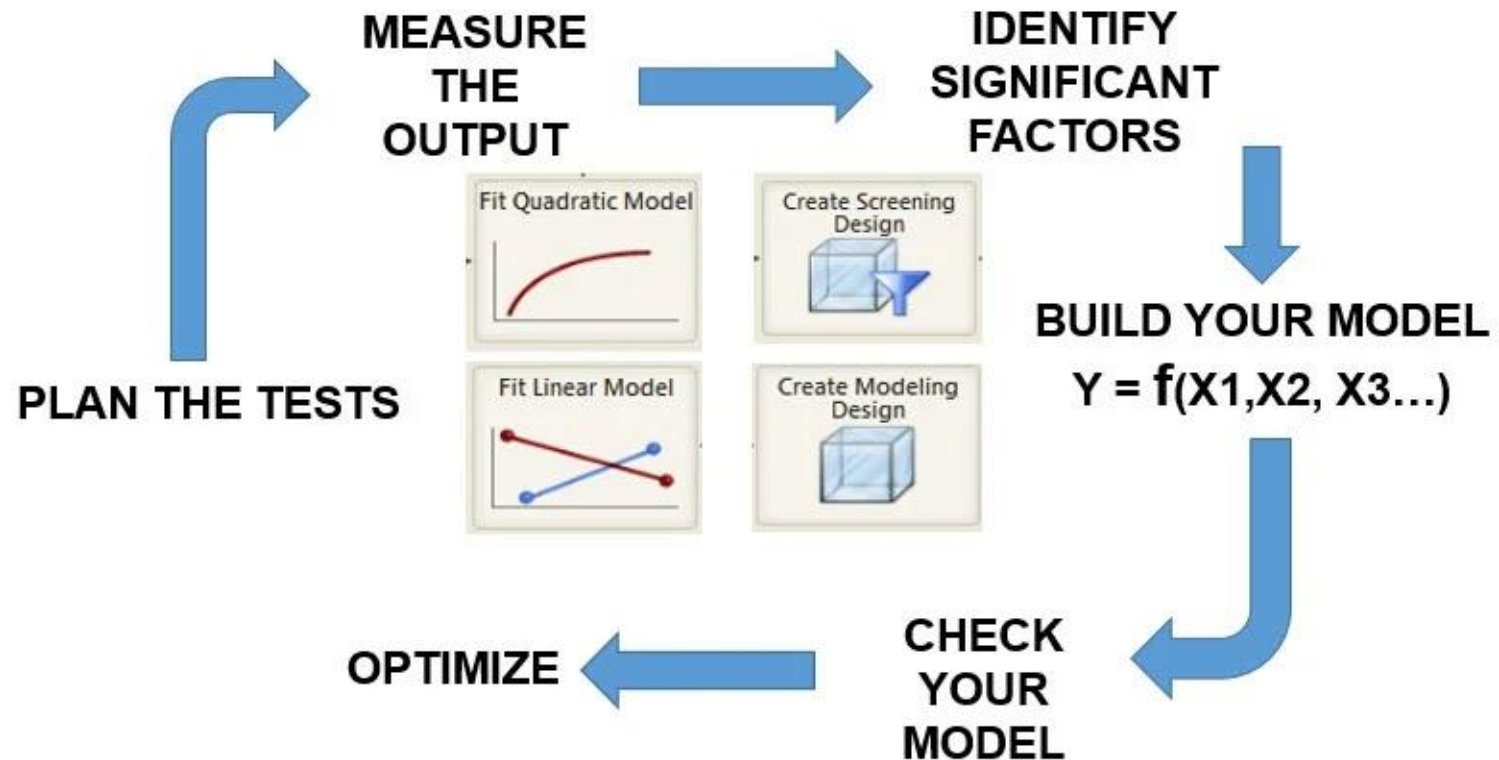


Key Concepts in Design of Experiments (DOE)

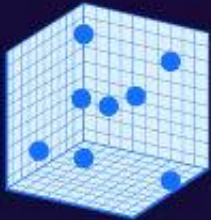
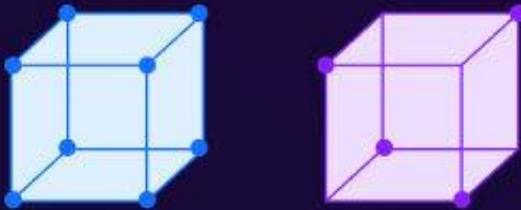
■ Factors and Levels

- **Factors:** Input variables that are being tested (e.g., mutation rate, population size in Genetic Algorithms).
- **Levels:** Specific values or settings for each factor (e.g., mutation rate = 0.1, 0.2, 0.3).

- **Responses:** The outputs or performance metrics affected by the factors (e.g., solution quality, execution time, convergence rate).
- **Experimental Runs:** Each combination of factor levels tested is an experimental run (e.g., running an algorithm with specific settings).



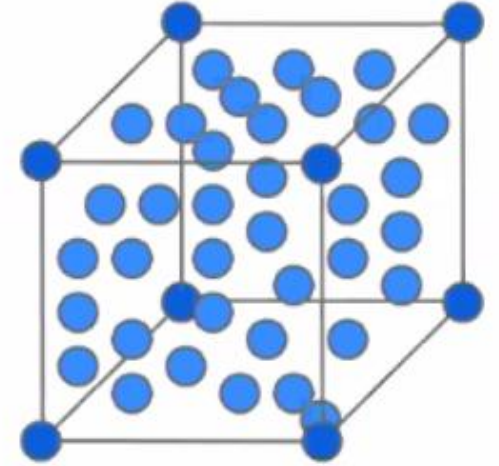
3 common types of DOE designs

	Space filling designs	Factorial designs	RSM designs
			
DOE stage	Scoping	Screening, Refinement and Iteration	Optimization, Robustness
Useful when...	You don't have much prior knowledge of your system	You're in the early stages of a campaign and are exploring lots of factors in outline	You only have a small number of (mostly) continuous factors
Useful for...	Investigating factors at lots of different levels	Refining factors during the first stages of a campaign before optimization	Finding the best levels for your system



Space Filling Designs

- **Purpose:** To explore the entire parameter space evenly.
- **Usefulness:**
 - Ideal when you lack prior knowledge about the system, allowing for a comprehensive understanding of how factors interact across the entire range.
 - Helps in identifying regions of interest in the parameter space.
- **Applications:** Often used in scenarios where the relationship between factors is not well understood, making it critical to sample widely across the input space.

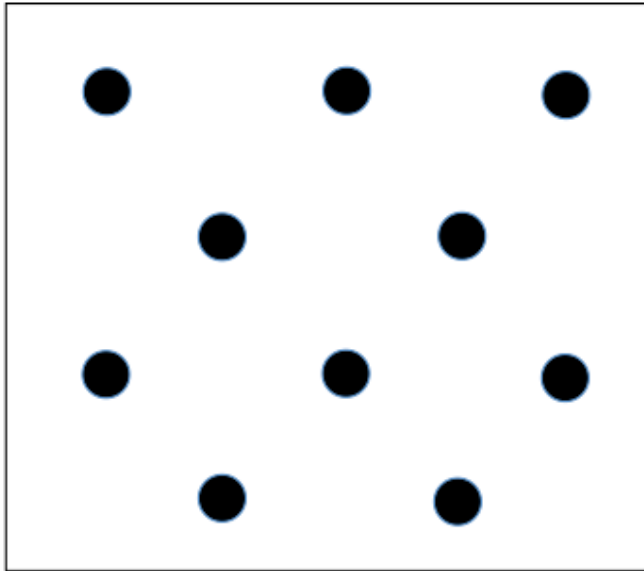


**Screening/Optimisation
Design**
Lack of prior knowledge
Non-Linear Landscapes



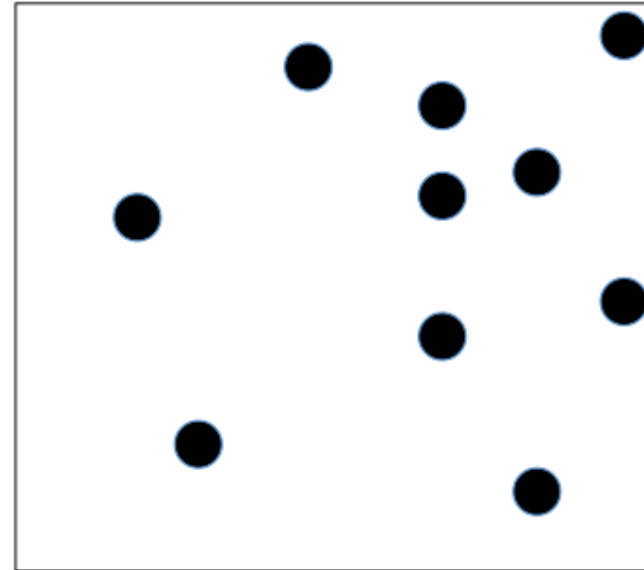
Space Filling Designs

Uniform Space Filling design



User provides:
- Candidate set of possible
input combinations

Non-Uniform Space Filling design

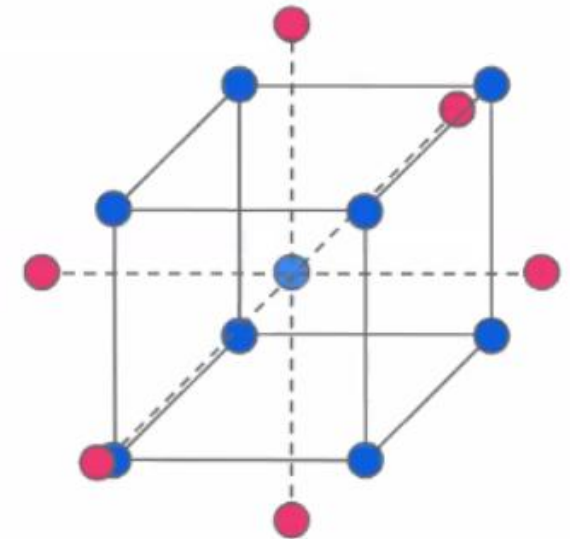


User provides:
- Candidate set of possible
input combinations with a
weight for each



Response Surface Methodology (RSM) Designs

- **Purpose:** To optimize a response variable by exploring the relationships between multiple explanatory variables and one or more response variables.
- **Usefulness:**
 - Best suited for cases where you have a small number of mostly continuous factors.
 - Aims to find the best levels for these factors to maximize or minimize the response.
- **Applications:** Frequently used in optimization problems, such as improving product quality or enhancing process efficiency, where a refined understanding of how factors influence outcomes is needed.



Response Surface Design

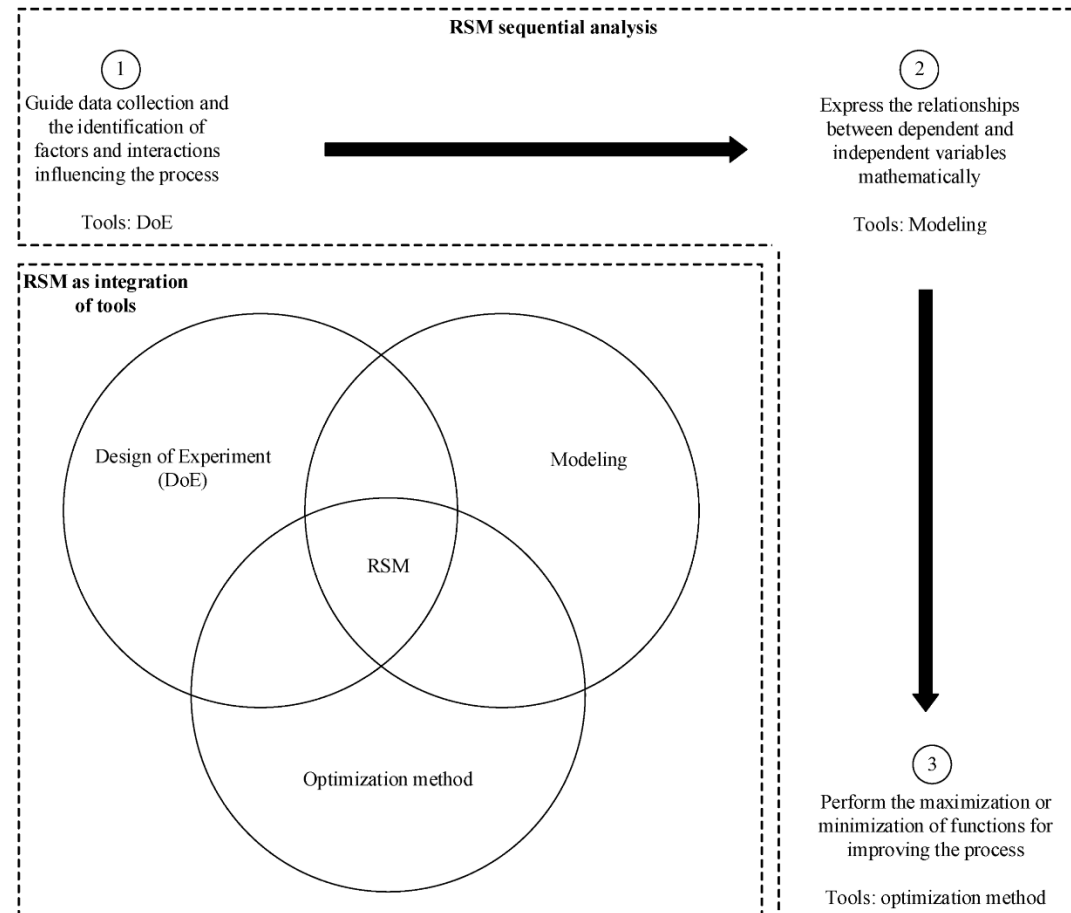
Optimisation

Characterisation of curvature

Detailed factor interactions



Response Surface Methodology (RSM) Designs

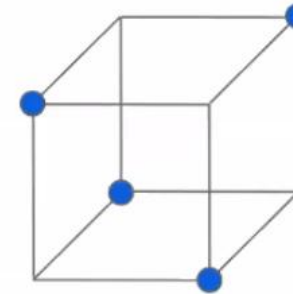




Factorial Design

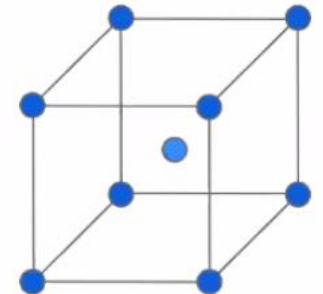
- **Purpose:** To assess the effects of multiple factors simultaneously by evaluating all possible combinations of factors at different levels.
- **Usefulness:**
 - Particularly useful in the early stages of a campaign when exploring many factors, enabling systematic investigation of interactions between variables.
 - Allows for efficient screening of significant factors that affect the response variable.
- **Applications:** Commonly applied in industrial experiments, product development, and other fields where understanding the impact of various factors is essential.

2-Level
Fractional Factorial



Screening Design
Factor Screening
Factor Interactions

2-Level
Fractional Factorial
+ Midpoints



Screening Design
Refining
Indication of curvature (Factor Interactions)

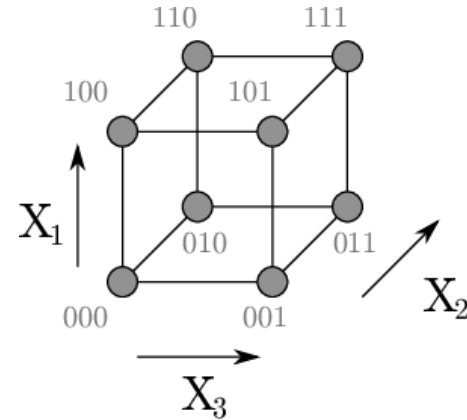


Fractional Factorial Designs

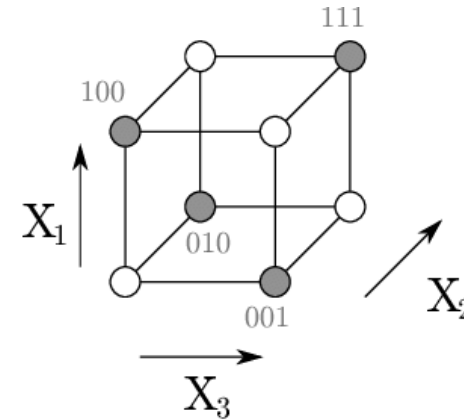
- Assuming the number of factors is k in an n level factorial design, then number of runs for even a single replicate of the n^k design becomes very large.
 - E.g, a replicate of an 8 factor two level experiment would require $2^8 = 256$ runs.
 - If a run consists of 31 trials each taking 5 min, such an experiment would take ~28 days (1 instance).
- **Fractional factorial designs** can be used in these cases to draw out valuable conclusions from fewer runs.
- **Key observation:** Responses are often affected by a small number of main effects and *lower order interactions*, while higher order interactions are relatively unimportant.



Fractional Factorial Designs



Full factorial



Fractional factorial

Run	Treatment	Factor			2-Way Interaction			3-Way Interaction
		A	B	C	AB	BC	AC	ABC
1	I	-1	-1	-1	+1	+1	+1	-1
2	a	+1	-1	-1	-1	+1	-1	+1
3	b	-1	+1	-1	-1	-1	+1	+1
4	ab	+1	+1	-1	+1	-1	-1	-1
5	c	-1	-1	+1	+1	-1	-1	+1
6	ac	+1	-1	+1	-1	-1	+1	-1
7	bc	-1	+1	+1	-1	+1	-1	-1
8	abc	+1	+1	+1	+1	+1	+1	+1



When Factorial Design is Not Possible

- Factorial designs evaluate all combinations of factors and levels, which can be impractical due to constraints like time, cost, or complexity.
- In such cases, alternative **sampling methods** are used to explore the parameter space without exhaustively testing every combination.



Design of Experiments – Sampling

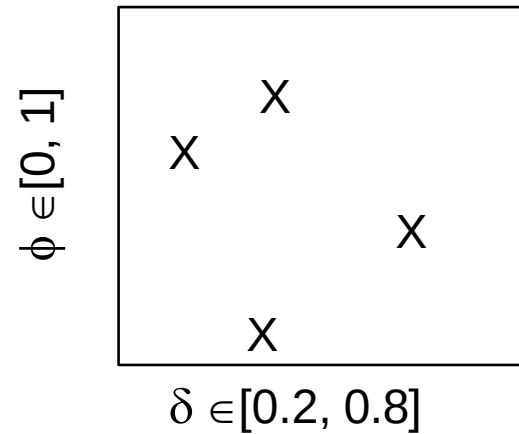
- Sampling Methods:
 - **Random**
 - **Latin Hyper-cube**
 - **Orthogonal**

- Example: Assume that we have two parameters,
 - $\phi \in [0, 1]$ and $\delta \in [0.2, 0.8]$



Random Sampling

- Generate each sample point independently (M).
- Example:





Random Sampling

- Involves selecting samples randomly from the entire parameter space.
- **Advantages:**
 - Simple and straightforward.
 - Can help avoid bias by ensuring that every point in the space has an equal chance of being selected.
- **Disadvantages:** May not provide comprehensive coverage of the parameter space, especially in high-dimensional problems.

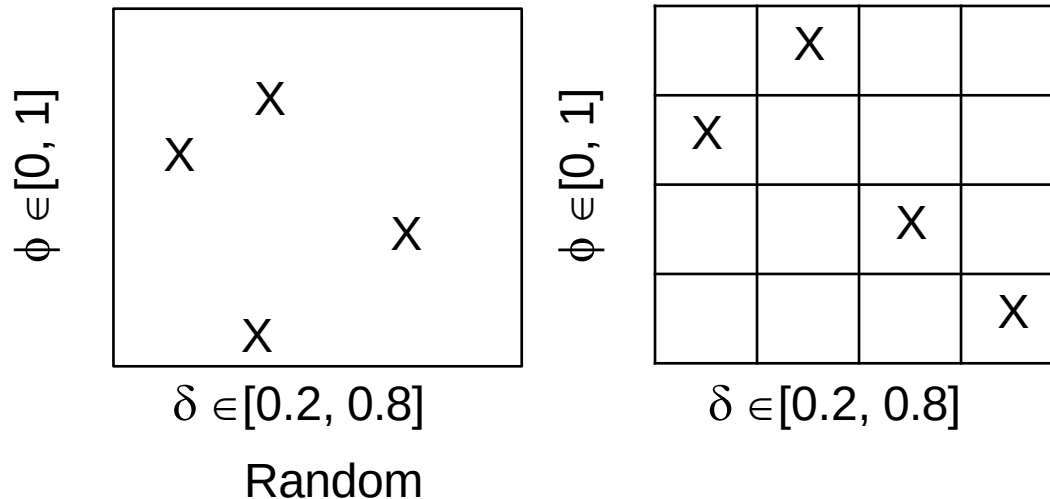
Example:

- Sample 1: $\phi=0.45, \delta=0.5$
- Sample 2: $\phi=0.12, \delta=0.75$
- Sample 3: $\phi=0.78, \delta=0.3$
- Sample 4: $\phi=0.33, \delta=0.65$



Latin Hypercube Sampling

- Decide the number of sample points (M) for N variables and for each sample point remember in which row and column the sample point was taken.
- Example:





Latin Hypercube Sampling (LHS)

- A statistical method that ensures that the *entire parameter space is sampled* more uniformly than random sampling.
 - The ranges of each parameter are divided into intervals based on the number of desired samples.
 - Samples are drawn randomly from each interval, ensuring that all intervals are represented.
- **Advantages:**
 - More efficient than simple random sampling in exploring the parameter space.
 - Reduces the chances of clustering, leading to better coverage of the space.



Example

Example:

- Divide ϕ into 4 intervals:
 - $[0, 0.25]$, $[0.25, 0.5]$, $[0.5, 0.75]$, $[0.75, 1]$
- Divide δ into 4 intervals:
 - $[0.2, 0.35]$, $[0.35, 0.5]$, $[0.5, 0.65]$, $[0.65, 0.8]$

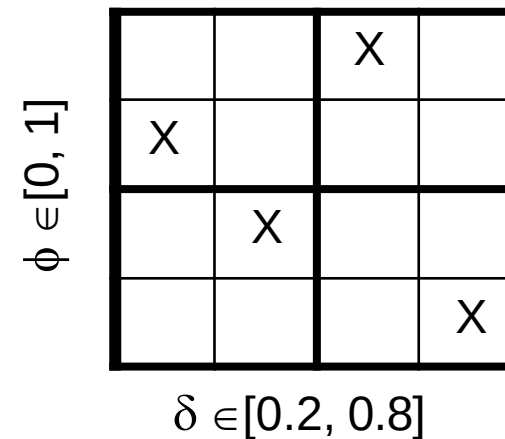
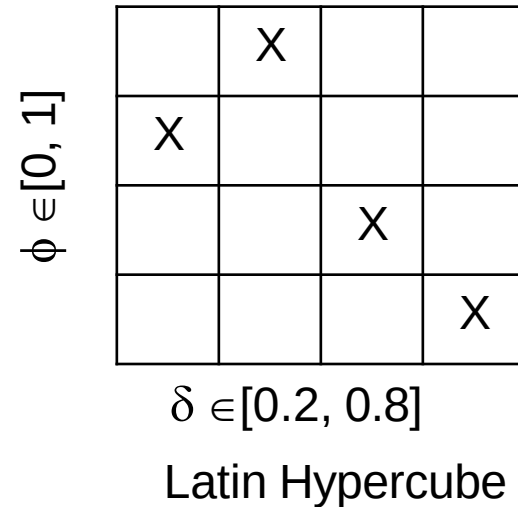
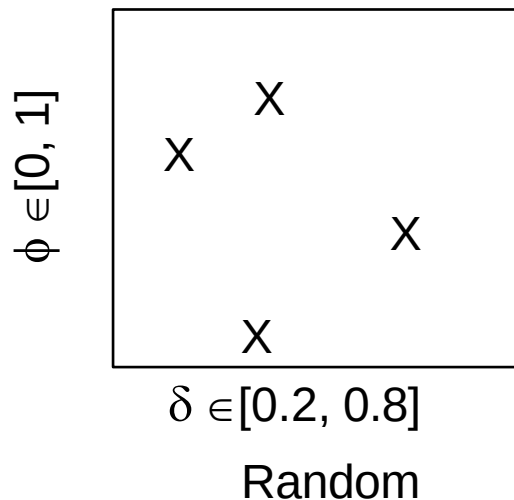
Then, randomly select one value from each interval:

- Sample 1: $\phi = 0.1$ (from $[0, 0.25]$), $\delta = 0.3$ (from $[0.2, 0.35]$)
- Sample 2: $\phi = 0.4$ (from $[0.25, 0.5]$), $\delta = 0.55$ (from $[0.5, 0.65]$)
- Sample 3: $\phi = 0.65$ (from $[0.5, 0.75]$), $\delta = 0.7$ (from $[0.65, 0.8]$)
- Sample 4: $\phi = 0.9$ (from $[0.75, 1]$), $\delta = 0.25$ (from $[0.2, 0.35]$)



Orthogonal Sampling

- The sample space is divided into equally probable subspaces. Sample points simultaneously, ensuring they form an ensemble of Latin Hypercube sample.
- Example:





Orthogonal Sampling

- Involves selecting samples in such a way that the sampling points are *orthogonal to each other*, maintaining independence between the parameters.
- **Advantages:**
 - Facilitates the analysis of variance (ANOVA) by ensuring that factors can be estimated independently.
 - Helps in minimizing correlations between factors, improving the reliability of results.



Example

Example:

- Choose a set number of points for each parameter (e.g., 4 points each):

For ϕ :

- 0.0, 0.33, 0.67, 1.0

For δ :

- 0.2, 0.4, 0.6, 0.8

Combine these in a way that each combination maintains orthogonality:

- Sample 1: $\phi = 0.0, \delta = 0.2$
- Sample 2: $\phi = 0.33, \delta = 0.4$
- Sample 3: $\phi = 0.67, \delta = 0.6$
- Sample 4: $\phi = 1.0, \delta = 0.8$



Taguchi Orthogonal Arrays Method

- Developed by Dr. Genichi Taguchi
- **Taguchi Orthogonal Arrays Method** focuses on improving quality and performance while minimizing variation.
- Taguchi method is a structured statistical (experimental design) method for determining the **best** combination of parameter settings to achieve certain objective(s). It uses **orthogonal arrays** to systematically test multiple factors (variables) with a minimal number of experiments. This method is particularly useful when testing all possible combinations of factors is impractical due to time or cost constraints.
- Aim: make a “product” or “process” less variable (more *robust*) in the face of variation over which we have little or no control.



Key Concepts of Taguchi's Method

- **Orthogonal Arrays (OAs):**

- These are predefined, balanced experimental designs that ensure factor levels are tested equally and independently of one another.
- The arrays are "orthogonal," meaning each factor's effect can be estimated independently of others.
- OAs reduce the number of experiments while maintaining statistical efficiency.

- **Signal-to-Noise (S/N) Ratio:**

- This metric evaluates the robustness of a system by quantifying the relationship between the desired signal (output) and the noise (variability caused by uncontrollable factors).
- Higher S/N ratios indicate more robust systems.
- Common S/N ratios:
 - Smaller-the-better: For minimizing responses (e.g., defects).
 - Larger-the-better: For maximizing responses (e.g., yield).
 - Nominal-the-best: For achieving a specific target value (e.g., dimension tolerance).



Key Concepts of Taguchi's Method

- **Control and Noise Factors:**
 - Control Factors: Variables that are manipulated in the experiment to optimize performance (e.g., temperature, pressure, algorithm parameters).
 - Noise Factors: Uncontrollable factors that introduce variability (e.g., environmental conditions).
- **Levels:** Each factor is tested at a few discrete levels (e.g., high, medium, low) to understand its effect on the response.
- **Robustness:** The Taguchi method focuses on making the design less sensitive to noise factors, improving consistency under real-world conditions.



Taguchi Orthogonal Arrays Method for Parameter Tuning – Main Steps

1. Selection of control parameters (independent variables/factors).
2. Selection of number of level settings for each parameter.
3. Select a suitable orthogonal array based on the number of parameters and levels.
4. Conduct the experiments using the algorithm on the selected subset of test instances.
5. Analyse the results.
6. Determine the optimum levels for the individual parameters.
7. Confirmation experiment.
Use the same configuration for the rest of the experiments

Planning

Conduct Analysis

Validation



Taguchi Orthogonal Arrays Method for Parameter Tuning

- Best when there are an intermediate number of parameters/variables/ factors (3 to 50), few interactions between variables, and when only a few variables contribute significantly.
 - E.g., 3 factors and 2 levels (settings) per factor: 2^3 combinations
- Taguchi's orthogonal arrays are highly fractional orthogonal designs, which can be used to estimate main effects using only a few experimental runs (which can consist of multiple trials). E.g.,

L4 (2^3)

Run	Columns		
	1	2	3
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1

L8 (2^7)

Run	Columns						
	1	2	3	4	5	6	7
1	1	1	1	1	1	1	1
2	1	1	1	2	2	2	2
3	1	2	2	1	1	2	2
4	1	2	2	2	2	1	1
5	2	1	2	1	2	1	2
6	2	1	2	2	1	2	1
7	2	2	1	1	2	2	1
8	2	2	1	2	1	1	2

L9 (3^4)

Run	Columns			
	1	2	3	4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1



Example of Taguchi Method

- **Problem:** Optimize the parameters of a machine for minimal defects.
- **Factors:**
 - Temperature (3 levels: Low, Medium, High)
 - Pressure (2 levels: Low, High)
 - Speed (2 levels: Low, High)
- **Orthogonal Array:** Use L_8 OA (8 experiments), which can accommodate 3 factors with up to 3 levels.



Example of Taguchi Method

Experiment	Temperature (A)	Pressure (B)	Speed (C)
1	Low	Low	Low
2	Low	Low	High
3	Low	High	Low
4	Low	High	High
5	Medium	Low	Low
6	Medium	Low	High
7	Medium	High	Low
8	Medium	High	High

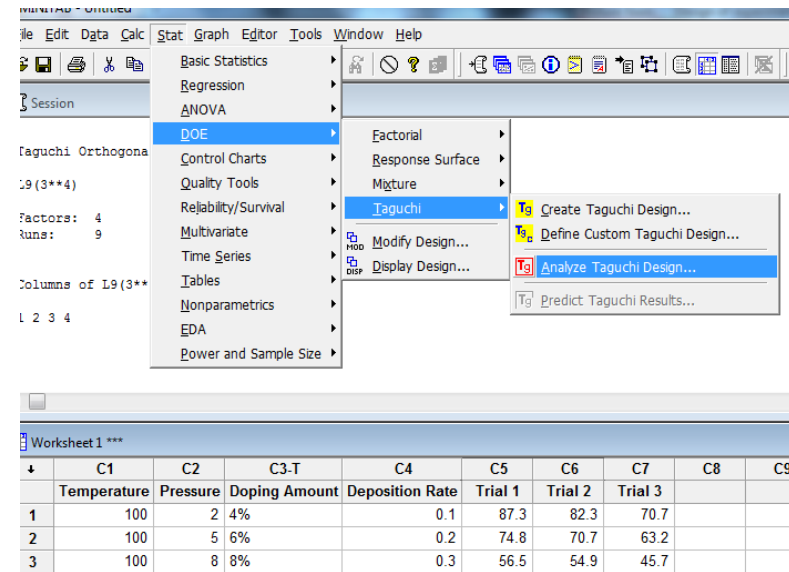
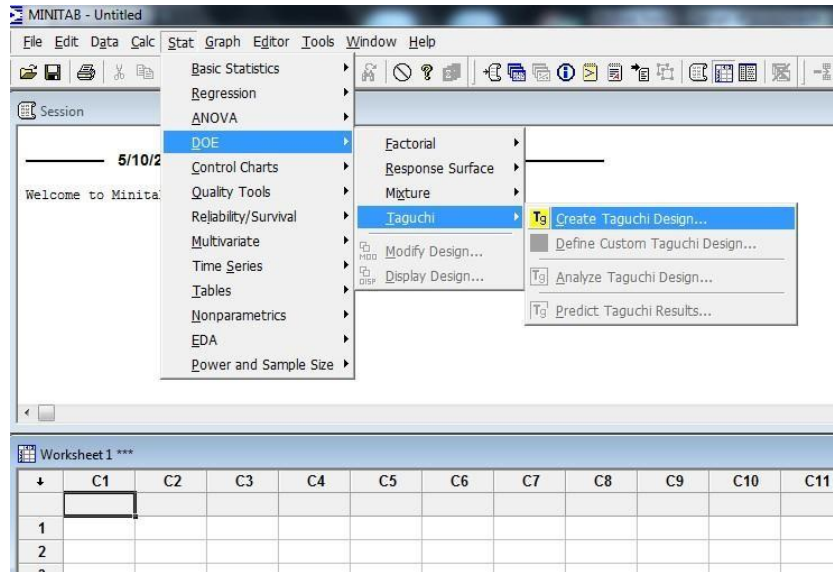


Example of Taguchi Method

- Steps:
 1. Conduct the 8 experiments with the specified combinations.
 2. Measure the number of defects (response).
 3. Calculate S/N ratios.
 4. Identify the best factor levels for minimizing defects.



Taguchi Orthogonal Arrays Method – Software Packages



Minitab

- MATLAB, Minitab, R, Octave, JMP, DOE++, ...



Parameter Tuning - Planning

- **Planning** refers to the systematic approach of *preparing* and *organizing experiments* or processes to identify the best possible parameter settings for a system or algorithm before it is executed.
- This is an **off-line** process, meaning all parameter adjustments are determined prior to the algorithm's or system's runtime.
- The goal of **planning** in parameter tuning is to efficiently explore the parameter space and find the combination of parameter values that optimizes performance (e.g., minimizes error, maximizes accuracy, or achieves other desired objectives).



Parameter Tuning - Planning

- Assume that we have a metaheuristic with 4 parameters (factors):

- param1 $\in [0, 1.0] (\mathbb{R})$
- param2 $\in [0, 1.0] (\mathbb{R})$
- param3 $\in [1..80] (\mathbb{Z}^+)$
- param4 $\in [1..5] (\mathbb{Z}^+)$

Parameter levels for memetic algorithm

Parameters	Value Options
	{0.2, 0.4, 0.6, 0.8, 1.0}
	{0.2, 0.4, 0.6, 0.8, 1.0}
	{5, 10, 20, 40, 80}
	{2, 3, 4, 5}

- Choosing a suitable Taguchi orthogonal array design:

- We have 4 factors with a maximum of 5 levels for each factor

- L4: Three two-level factors
- L8: Seven two-level factors
- L9: Four three-level factors
- L12: Eleven two-level factors
- L16: Fifteen two-level factors
- L16b: Five four-level factors
- L18: One two-level and seven three-level factors
- L25: Six five-level factors
- L27: Thirteen three-level factors
- L32: Thirty-two two-level factors

X1 X2 X3 X4 X5 X6
 - - - - -
 1 1 1 1 1 1
 1 2 2 2 2 2
 1 3 3 3 3 3
 1 4 4 4 4 4
 1 5 5 5 5 5
 2 1 2 3 4 5
 2 2 3 4 5 1
 2 3 4 5 1 2
 2 4 5 1 2 3
 2 5 1 2 3 4
 3 1 3 5 2 4
 3 2 4 1 3 5
 3 3 5 2 4 1
 3 4 1 3 5 2
 3 5 2 4 1 3
 4 1 4 2 5 3
 4 2 5 3 1 4
 4 3 1 4 2 5
 4 4 2 5 3 1
 4 5 3 1 4 2
 5 1 5 4 3 2
 5 2 1 5 4 3
 5 3 2 1 5 4
 5 4 3 2 1 5
 5 5 4 3 2 1

Taguchi orthogonal array

Experiment	param1	param2	param3	param4	size
1	1	1	1	1	1
2	1	2	2	2	2
3	1	3	3	3	3
4	1	4	4	4	4
5	1	5	5	5	5
6	2	1	2	3	6
7	2	2	3	4	7
8	2	3	4	5	8
9	2	4	5	1	9
10	2	5	1	2	10
11	3	1	3	4	11
12	3	2	4	5	12
13	3	3	5	1	13
14	3	4	1	2	14
15	3	5	2	3	15
16	4	1	4	5	16
17	4	2	5	1	17
18	4	3	1	2	18
19	4	4	2	3	19
20	4	5	3	4	20
21	5	1	5	1	21
22	5	2	1	2	22
23	5	3	2	3	23
24	5	4	3	4	24
25	5	5	4	5	25



Suitable Taguchi Orthogonal Array Design

1. Parameters and Their Levels:

1. param1: Continuous (0 to 1)
2. param2: Continuous (0 to 1)
3. param3: Discrete (1 to 80)
4. param4: Discrete (1 to 5)

2. Number of Factors: 4 factors

3. Maximum Levels: Up to 5 levels for each factor

- For 4 factors with a maximum of 5 levels, you can use the **L25 (5^6)** orthogonal array, which allows for 5 levels for each of the 4 factors. However, since you have 6 degrees of freedom in the L25 array, you can only fully utilize it up to 5 factors.



Suitable Taguchi Orthogonal Array

- **Continuous Parameters:** For continuous parameters (param1 and param2), you can assign specific values based on your experimentation needs (e.g., 0.0, 0.2, 0.4, 0.6, 0.8, 1.0).
- **Discrete Parameters:** For discrete parameters (param3 and param4), you can assign integers within the specified ranges.

Example of L25 Array

The L25 array is structured as follows:

Trial	param1	param2	param3	param4
1	0.2	0.2	1	1
2	0.2	0.4	2	2
3	0.2	0.6	3	3
4	0.2	0.8	4	4
5	0.2	1.0	5	5
6	0.4	0.2	1	2
7	0.4	0.4	2	3
8	0.4	0.6	3	4
9	0.4	0.8	4	5
10	0.4	1.0	5	1
...	...	...	...	...
25	1.0	1.0	5	5



Planning - Example

Parameter levels

		Value Options
[5 levels]	param1	{0.2, 0.4, 0.6, 0.8, 1.0}
[5 levels]	param2	{0.2, 0.4, 0.6, 0.8, 1.0}
[5 levels]	param3	{5, 10, 20, 40, 80}
[4 levels]	param4	{2, 3, 4, 5}
		1 2 3 4

Taguchi orthogonal array

Configuration ID		param1	param2	param3	param4
1	e.g.;	1 (0.2)	1 (0.2)	1 (5)	1 (2)
2		1	2	2	2
3		1	3	3	3
4		1	4	4	4
5		1	5	5	2
6		2	1	2	3
7		2	2	3	4
8		2	3	4	1
9		2	4	5	1
10		2	5	1	2
11		3	1	3	4
12		3	2	4	1
13		3	3	5	2
14		3	4	1	3
15		3	5	2	4
16		4	1	4	2
17		4	2	5	3
18		4	3	1	4
19		4	4	2	2
20		4	5	3	1
21		5	1	5	4
22		5	2	1	3
23		5	3	2	1
24		5	4	3	2
25	e.g.;	5 (1.0)	5 (1.0)	4 (40)	3 (4)



Conduct Experiments - Example

- Run experiments, say for 30 times using the algorithm with each setting (potentially on multiple ‘training’ instances).
- Use a performance metric, e.g., record Formula 1 score for each run/trial (higher the better), where the top 8 algorithms score **10, 8, 6, 5, 4, 3, 2** and **1** point(s).

Configuration ID	param1	param2	param3	param4
1	0.2	0.2	5	2
2	0.2	0.4	10	3
3	0.2	0.6	20	4
4	0.2	0.8	40	5
5	0.2	1.0	80	3
6	0.4	0.2	10	4
7	0.4	0.4	20	5
8	0.4	0.6	40	2
9	0.4	0.8	80	2
10	0.4	1.0	5	3
11	0.6	0.2	20	5
12	0.6	0.4	40	2
13	0.6	0.6	80	3
14	0.6	0.8	5	4
15	0.6	1.0	10	5
16	0.8	0.2	40	3
17	0.8	0.4	80	4
18	0.8	0.6	5	5
19	0.8	0.8	10	3
20	0.8	1.0	20	2
21	1.0	0.2	80	5
22	1.0	0.4	5	4
23	1.0	0.6	10	2
24	1.0	0.8	20	3
25	1.0	1.0	40	4



Conduct Experiments - Example

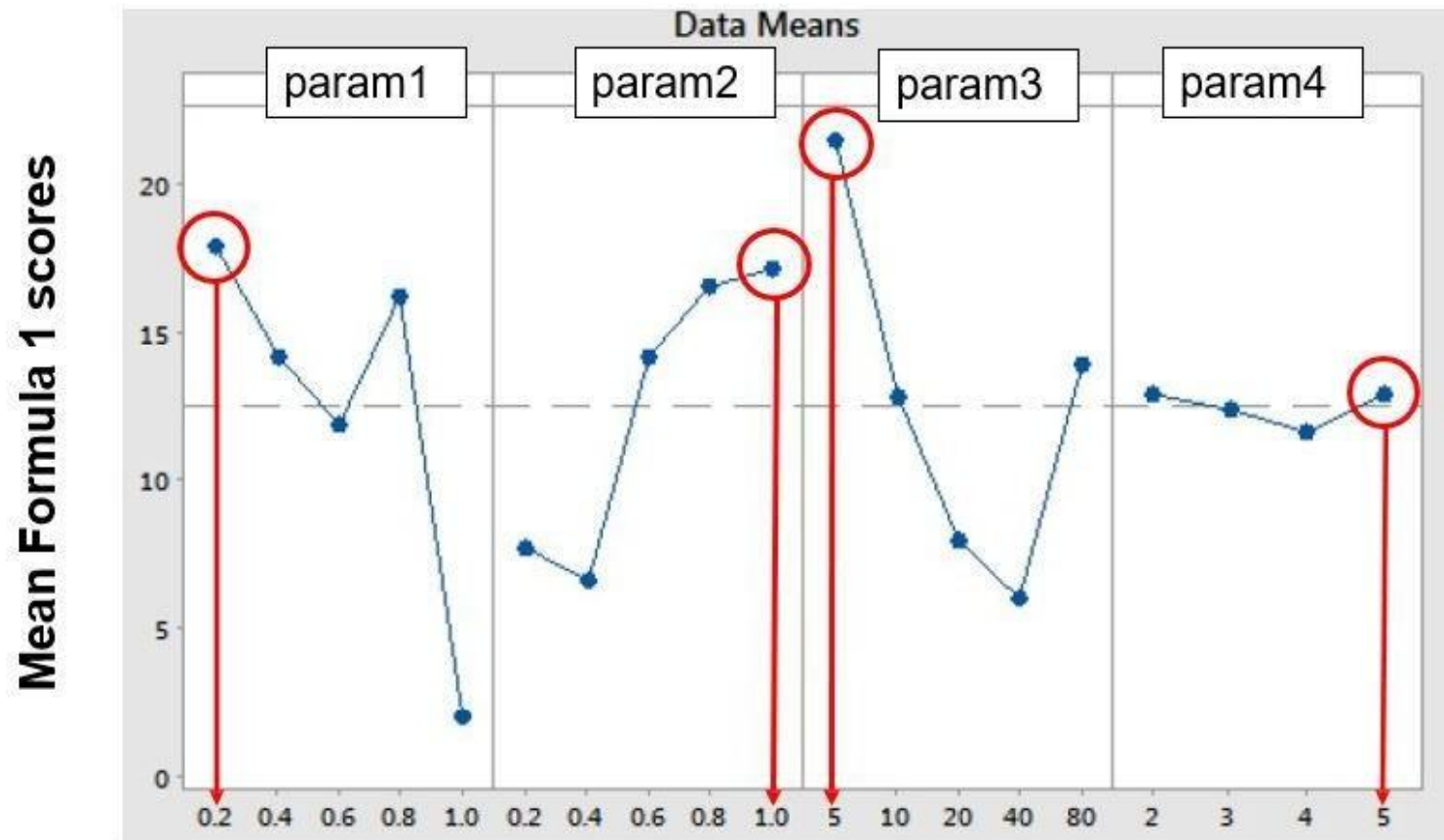
- Collect results, e.g., mean F1 score per run for each algorithm with a specific parameter combination setting over the instances.
- For analysis: Main effect of param3 for the setting of 20 is the mean scores for all experiments with that setting per run:

$$(17.5+2.5+0+18.38+1.88) / 5 = 8.052$$

Configuration ID	param1	param2	param3	param4	mean F1 score
1	0.2	0.2	5	2	24.88
2	0.2	0.4	10	3	14
3	0.2	0.6	20	4	17.50
4	0.2	0.8	40	5	17.88
5	0.2	1.0	80	3	15.50
6	0.4	0.2	10	4	9.88
7	0.4	0.4	20	5	2.50
8	0.4	0.6	40	2	8.88
9	0.4	0.8	80	2	22.50
10	0.4	1.0	5	3	27.25
11	0.6	0.2	20	5	0
12	0.6	0.4	40	2	1
13	0.6	0.6	80	3	12
14	0.6	0.8	5	4	24.38
15	0.6	1.0	10	5	22.25
16	0.8	0.2	40	3	0
17	0.8	0.4	80	4	25.88
18	0.8	0.6	5	5	30.88
19	0.8	0.8	10	3	16.25
20	0.8	1.0	20	2	18.38
21	1.0	0.2	80	5	4
22	1.0	0.4	5	4	0
23	1.0	0.6	10	2	1.88
24	1.0	0.8	20	3	1.88
25	1.0	1.0	40	4	2.50



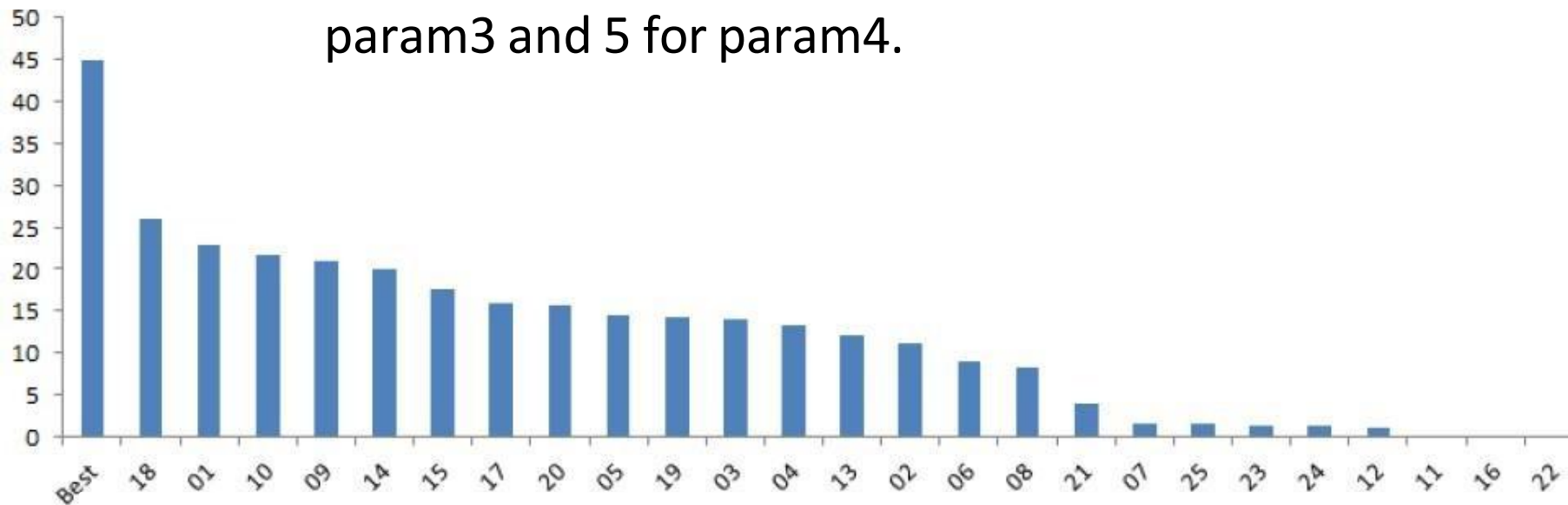
Analysis – Main Effects Plot





Validation: Confirmation Run

Best configuration: 0.2 for param1, 1.0 for param2, 5 for param3 and 5 for param4.



If failure, then chosen levels could be problematic needs to be reviewed.



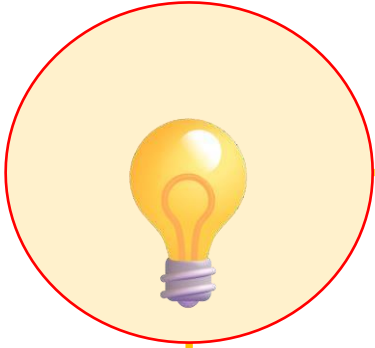
Summary

- It is not trivial *which (meta)heuristic optimisation/search algorithm will perform better than the other one* on a given problem domain.
 - Experiments using a single or only *small instances might not* be realistic or true performance indicator of an algorithm.
- There is a variety of *statistical tools* for the performance analyses of algorithms.



Summary

- Many (meta)heuristic optimisation/search algorithms come with parameters which often require an initial setting (tuning).
 - **Parameter tuning** is possible; however, it is time consuming.
 - There is a range of different techniques varying from manual/semi-automated experimental design methods to automated tuning, such as, Taguchi method, I-race.
 - **Parameter control** as an alternative to parameter tuning changes parameter values during the run of the algorithm.
 - There is no guidance indicating which method is the best, however many studies show that parameter tuning/control often does improve the performance of an algorithm as compared to the variant where it is not used.



1. Move Acceptance
2. Simulated Annealing
3. **Parameter Setting Issues**





University of
Nottingham

UK | CHINA | MALAYSIA

NEXT:

**Evolutionary
Algorithm I**